OS WEEK 4

1. **rdfifo.c**

- Membuat named pipe (FIFO) bernama fifo1
- Membuka FIFO tersebut dalam mode read-only
- Membaca data dari FIFO
- Menampilkan data yang dibaca ke layar
- Menghapus FIFO setelah selesai

Jadi program ini adalah program penerima (reader) menggunakan FIFO. Program lain harus menulis ke fifo1, baru program ini bisa membaca.

```c
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <limits.h>

/*
sys/types.h → tipe data sistem
sys/stat.h → fungsi untuk file (termasuk mkfifo)
unistd.h → fungsi read, close
errno.h → untuk menangani error
stdio.h → printf
stdlib.h → exit
fcntl.h → open dan flag seperti O_RDONLY
limits.h → untuk ukuran buffer PIPE_BUF
*/

int main(void)
{

int fd;
int len;
char buf[PIPE_BUF];
mode_t mode = 0666;

/*
fd → file descriptor (penanda file)
len → menyimpan jumlah byte yang dibaca
buf[PIPE_BUF] → buffer untuk menyimpan data dari FIFO
mode = 0666 → permission file (read & write untuk semua user)*/
```

```c
// Membuat named pipe bernama "fifo1"
if ((mkfifo("fifo1", mode)) < 0)
    {
        // Menyimpan kode error jika mkfifo gagal
        int error = errno;

        // Jika error karena FIFO sudah ada
        if (error == EEXIST)
            // Lanjut pakai FIFO yang sudah ada
            printf("Continue using an existing pipe\n");
        // Jika error selain FIFO sudah ada
        else
        {
            // Tampilkan pesan error dari sistem
            perror("mkfifo");
            // Keluar dari program dengan status gagal
            exit(EXIT_FAILURE);
        }
    }

    // Membuka FIFO dalam mode read-only
    if ((fd = open("fifo1", O_RDONLY)) < 0)
    {
        // Tampilkan pesan error jika gagal membuka
        perror("open");
        // Keluar dari program dengan status gagal
        exit(EXIT_FAILURE);
    }

    // Membaca data dari FIFO selama masih ada data
    while ((len = read(fd, buf, PIPE_BUF - 1)) > 0)
    {
        // Menambahkan null terminator agar aman dicetak sebagai string
        buf[len] = '\0';
        // Menampilkan data yang dibaca ke layar
        printf("fifo read: %s", buf);
    }

    // Menutup file descriptor FIFO
    close(fd);
    // Menghapus file FIFO dari filesystem
    remove("fifo1");
    // Keluar dari program dengan status sukses
    exit(EXIT_SUCCESS);

}
```

2. **wrfifo.c**
    - Membuka FIFO bernama fifo1 dalam mode write-only
    - Setiap 3 detik:
        i. Mengambil waktu saat ini
        ii. Membuat string berisi PID dan waktu
        iii. Mengirim string itu ke FIFO

    - Terus berjalan tanpa henti (loop selamanya)

    Jadi ini adalah program pengirim(sender) data ke FIFO.

```c
// Menggunakan tipe data sistem seperti mode_t dan pid_t
#include <sys/types.h>

// Untuk operasi file dan FIFO
#include <sys/stat.h>

// Untuk fungsi sistem seperti write(), close(), sleep(), getpid()
#include <unistd.h>

// Untuk menangani error sistem
#include <errno.h>

// Untuk fungsi input output seperti printf(), sprintf(), perror()
#include <stdio.h>

// Untuk fungsi exit() dan konstanta EXIT_SUCCESS / EXIT_FAILURE
#include <stdlib.h>

// Untuk fungsi open() dan flag seperti O_WRONLY
#include <fcntl.h>

// Untuk konstanta PIPE_BUF (ukuran buffer maksimum pipe)
#include <limits.h>

// Untuk fungsi waktu seperti time() dan ctime()
#include <time.h>

// Fungsi utama program
int main(void)
{
    // File descriptor untuk FIFO
    int fd;

    // Menyimpan jumlah byte yang ditulis
    int len;

    // Buffer untuk menyimpan pesan yang akan dikirim
    char buf[PIPE_BUF];

    // Variabel untuk menyimpan waktu saat ini
    time_t tp;
```

```c
    // Membuka FIFO bernama "fifo1" dalam mode write-only
    // Jika tidak ada reader, maka open() akan menunggu
    if ((fd = open("fifo1", O_WRONLY)) < 0)
    {
        // Menampilkan pesan error jika gagal membuka FIFO
        perror("open");

        // Keluar dari program dengan status gagal
        exit(EXIT_FAILURE);
    }

    // Loop tanpa henti untuk terus mengirim data
    while (1)
    {
        // Mengambil waktu saat ini dan menyimpannya ke tp
        time(&tp);

        // Membuat string pesan berisi PID dan waktu sekarang
        // getpid() mengambil ID proses
        // ctime() mengubah waktu menjadi string yang bisa dibaca
        len = sprintf(buf, "fifo %d sends %s", getpid(), ctime(&tp));

        // Menulis isi buffer ke FIFO
        // len+1 agar karakter null '\0' ikut terkirim
        if ((write(fd, buf, len + 1)) < 0)
        {
            // Jika terjadi error saat menulis
            perror("write");

            // Tutup FIFO
            close(fd);

            // Keluar dengan status gagal
            exit(EXIT_FAILURE);
        }

        // Menunggu 3 detik sebelum mengirim pesan berikutnya
        sleep(3);
    }

    // Menutup file descriptor (tidak akan tercapai karena loop tak berhingga)
    close(fd);

    // Keluar dari program dengan status sukses
    exit(EXIT_SUCCESS);
}
```

Writer:

- Buka FIFO
- Ambil waktu
- Buat pesan
- Kirim pesan
- Tunggu 3 detik
- Ulang

Reader:

- Buka FIFO
- Baca pesan
- Tampilkan ke layar

3. **shmdemo.c**
    - Membuat atau mengakses shared memory ukuran 1024 byte

- Jika diberi argumen → menulis ke shared memory
- Jika tidak diberi argumen → membaca dari shared memory
- Melepas shared memory setelah selesai

Jadi satu file ini bisa jadi writer dan reader.

```c
// Untuk fungsi input output seperti printf dan fprintf
#include <stdio.h>

// Untuk exit()
#include <stdlib.h>

// Untuk fungsi string seperti strncpy()
#include <string.h>

// Untuk tipe data sistem
#include <sys/types.h>

// Untuk key IPC
#include <sys/ipc.h>

// Untuk fungsi shared memory seperti shmget dan shmat
#include <sys/shm.h>

// Mendefinisikan ukuran shared memory 1024 byte (1 KB)
#define SHM_SIZE 1024

// Fungsi utama dengan argumen command line
int main(int argc, char *argv[])
{
    // Variabel untuk key shared memory
    key_t key;

    // ID shared memory dan permission
    int shmid;

    // Pointer untuk mengakses shared memory
    char *data;

    // Jika argumen lebih dari 2 maka salah penggunaan
    if (argc > 2) {
        fprintf(stderr, "usage: ./shmdemo <data string>\n");
        exit(1);
    }

    // Membuat key unik berdasarkan file shmdemo.c dan karakter 'R'
    key = ftok("shmdemo.c", 'R');

    // Membuat atau mengakses shared memory ukuran 1024 byte
    // 0644 adalah permission
    // IPC_CREAT berarti buat jika belum ada
    shmid = shmget(key, SHM_SIZE, 0644 | IPC_CREAT);

    // Attach shared memory ke address space proses
    data = (char *) shmat(shmid, (void *)0, 0);

    // Jika ada argumen (argc == 2)
    // Maka tulis ke shared memory
    if (argc == 2) {
        printf("write: \"%s\"\n", argv[1]);

        // Salin string ke shared memory
        strncpy(data, argv[1], SHM_SIZE);
    }
    // Jika tidak ada argumen
    // Maka baca isi shared memory
    else {
        printf("read: \"%s\"\n", data);
    }

    // Detach shared memory dari proses
    shmdt(data);

    return 0;
}
```

4. **shemdemo_varian.c**

   Try changing the key values to, for example, ftok("shmdemo.c",'B') and ftok(".",'A'), recompile, and run demoshm. Observe the shared memory segment with the ipcs –m command.

5. **shmerase.c**
   - Menerima shmid dari command line
   - Melepas (detach) shared memory
   - Menghapus shared memory dari sistem

     Alternatif dari ipcrm -m <shmid>

     cara jalankannya:
     ./shmerase <shmid>

```c
// Untuk fungsi printf dan fprintf
#include <stdio.h>

// Untuk exit()
#include <stdlib.h>

// Untuk fungsi string
#include <string.h>

// Untuk tipe data sistem
#include <sys/types.h>

// Untuk IPC key dan struktur
#include <sys/ipc.h>

// Untuk fungsi shared memory seperti shmctl
#include <sys/shm.h>

// Fungsi utama dengan argument command line
int main(int argc, char *argv[])
{
    // Variabel untuk menyimpan shared memory ID
    int shmid;

    // Jika jumlah argumen bukan 2, tampilkan cara penggunaan
    if(argc != 2) {
        fprintf(stderr, "usage: %s <shmid>\n", argv[0]);
        exit(1);
    }

    // Mengubah argumen string menjadi integer (shmid)
    shmid = atoi(argv[1]);

    // Menghapus shared memory dengan perintah IPC_RMID
    shmctl(shmid, IPC_RMID, (struct shmid_ds *) 0);

    return 0;
}
```

6. **shmrace.c**

- Membuat shared memory untuk menyimpan satu integer
- Mengatur nilainya ke 0
- Melakukan fork() (membuat proses child)
- Parent dan child sama-sama menambah nilai integer sebanyak 100000 kali
- Menampilkan nilainya
- Menghapus shared memory

Program ini menunjukkan bahwa tanpa sinkronisasi, hasil akhirnya bisa tidak sesuai harapan.

```
// Untuk printf
#include <stdio.h>

// Untuk exit()
#include <stdlib.h>

// Untuk tipe data sistem
#include <sys/types.h>

// Untuk shared memory
#include <sys/shm.h>

// Untuk IPC
#include <sys/ipc.h>

// Untuk fork()
#include <unistd.h>

// Untuk waitpid()
#include <sys/wait.h>

// Jumlah perulangan
#define NLOOPS 100000

int main()
{
    // Variabel loop, PID child, shared memory id, key, dan pointer ke data
    int i, child, shmid, key = 100, *ptrData;

    // Membuat shared memory ukuran integer
    shmid = shmget(key, sizeof(int), 0644 | IPC_CREAT);

    // Attach shared memory ke proses
    ptrData = (int *) shmat(shmid, (void*)0, 0);

    // Inisialisasi nilai shared memory menjadi 0
    *ptrData = 0;

    // Membuat proses child
    child = fork();
```

```c
// Jika ini proses child
if(child == 0)
{
    // Loop sebanyak 100000 kali
    for(i = 0; i < NLOOPS; i++)
    {
        // Tambah nilai shared memory
        (*ptrData)++;

        // Tampilkan nilai
        printf("i=%d child: ptrData's value=%d\n", i, *ptrData);
    }
}
// Jika ini proses parent
else if(child > 0)
{
    for(i = 0; i < NLOOPS; i++)
    {
        // Tambah nilai shared memory
        (*ptrData)++;

        // Tampilkan nilai
        printf("i=%d parent: ptrData's value=%d\n", i, *ptrData);
    }

    // Menunggu child selesai
    int status;
    waitpid(child, &status, 0);
}

// Melepas shared memory
shmdt(ptrData);

// Menghapus shared memory dari sistem
shmctl(shmid, IPC_RMID, (struct shmid_ds *)0);

return 0;
}
```